



VAAAL UNIVERSITY
OF TECHNOLOGY

Inspiring thought. Shaping talent.

FACULTY: Applied and Computer Sciences
DEPARTMENT: Computer Sciences
SUBJECT: Advanced Software Design
SUBJECT CODE: ASSDX4A
LECTURER: Mr T Shongwe
MODERATOR:
YEAR: 2026

ASSESSMENT NAME:	ASSDX4A DEVOPS PRACTICAL ASSESSMENT REPORT
ASSESSMENT DATE:	22 May 2026

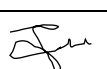
STUDENT NUMBER	SURNAME AND INITIALS	ROLE	SIGNATURE
223019267	Ravele W	Git Leader	
223430145	Shaun S	CI Engineer	
224062565	Motlatso M	Deployment Lead	
222476400	Phalane RM	Bug Fix Engineer	
22379250	Tshidiso T	QA/Tester	

Table of Contents

1. Introduction.....	3
2. Application Description.....	3
3. Objectives of the Project.....	4
4. Version Control and Collaboration.....	4
4.1 GitHub Repository (Screenshot).....	5
4.2 Feature Branches (Screenshot).....	6
4.3 Pull Requests (Screenshot).....	6
5. Build and Testing Process.....	7
5.1 Maven Build Output (Screenshot).....	7
6. Bug Identification and Resolution.....	8
6.1 Bug Report Table.....	8
6.2 Before Bug Fixes (Screenshot).....	9
6.3 After Bug Fixes (Screenshot).....	10
7. GitHub Actions Continuous Integration.....	11
7.1 GitHub Actions Red Build.....	11
7.2 GitHub Actions Green Build.....	12
8. Jenkins Pipeline Implementation.....	12
8.1 Jenkins Dashboard (Screenshot).....	13
8.2 Jenkins Console Output (Screenshot).....	13
9. Docker Containerisation.....	14
9.1 Docker Build.....	14
9.2 Running Docker Container.....	14
10. Deployment to Render.com.....	15
10.1 Render Dashboard (Screenshot).....	15
10.2 Live Running Application (Screenshot).....	16
11. Conclusion.....	17

1. Introduction

The goal of this practical assessment was to build a full complete Continuous Integration and Continuous Deployment (CI/CD) pipeline for a java web application called the student Grade Calculator. The assessment helped our group learn and apply DevOps tools and practices to automate software development tasks such as building, testing, integration, deployment, and teamwork.

During the assessment, the group used Git and GitHub for version control and collaboration, set up Continuous Integration using GitHub Actions and Jenkins, created automated tests to find bugs in the application, and fixed errors in the source code. The application was also containerised using Docker and deployed online using Render.com.

The group worked together using feature branches and pull requests. Overall, the project provided practical experience with modern DevOps tools and automated software delivery processes.

2. Application Description

The Student Grade Calculator is a web application that helps figure out a student's grade. It does this by using the grades from the semester and the exam. The Student Grade Calculator uses rules to decide what the student's final result will be like if they get a distinction or if they pass or fail. It also makes sure the grades people enter are real and between 0 and 100.

The Student Grade Calculator was made with Java and other tools, like Java Servlets and JavaServer Pages. Apache Tomcat and Maven were also used to build the student Grade Calculator. The Student Grade Calculator was working okay. It had some mistakes in the way it was working that made it give wrong answers. These mistakes were found when Student Grade Calculator was tested. They were fixed while the project was going on.

3. Objectives of the Project

The main objectives of this project were:

- To build a full CI/CD pipeline
- To automate testing and deployment of the application
- To improve teamwork using GitHub
- To find and fix bugs using unit tests
- To package the application using Docker
- To deploy the application to the cloud
- To gain practical experience with DevOps tools and methods

4. Version Control and Collaboration

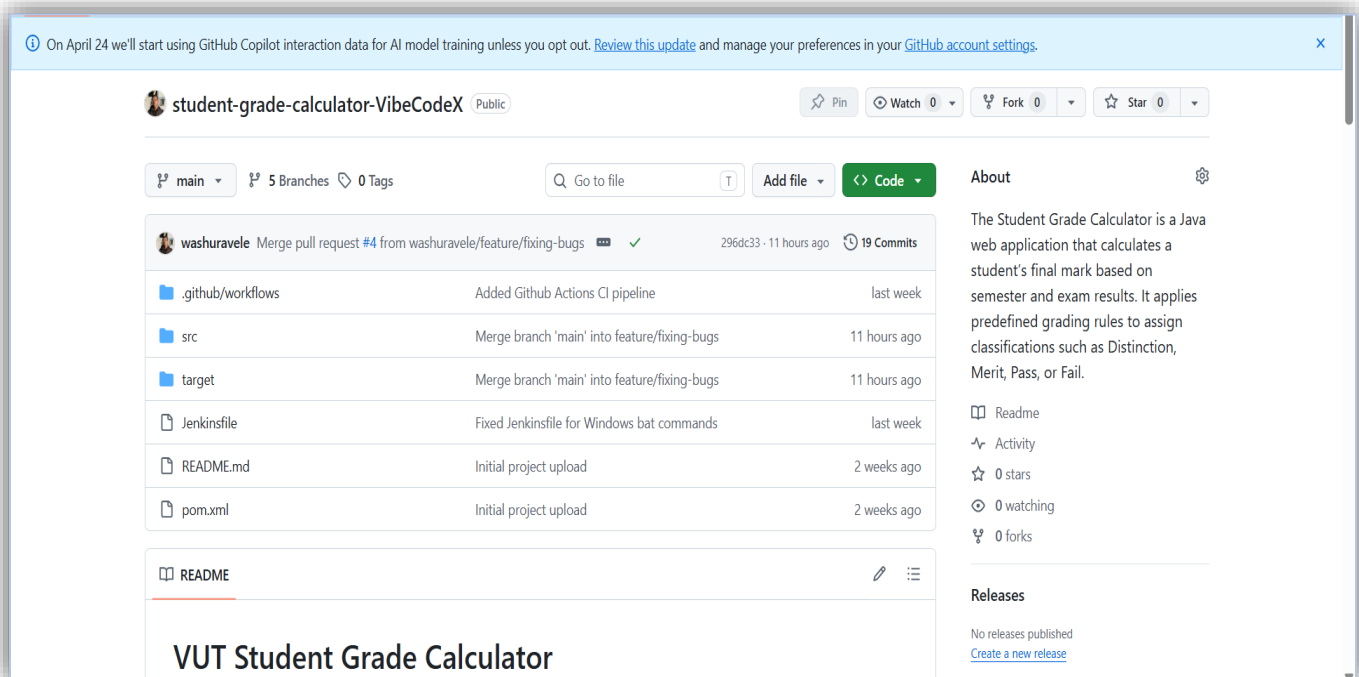
GitHub was used to manage the project code. The group leader created the repository and added all members. Each member copied the project to their own computer and worked on a separate feature branch. This allowed everyone to work on their tasks without changing the main code.

The following branches were created:

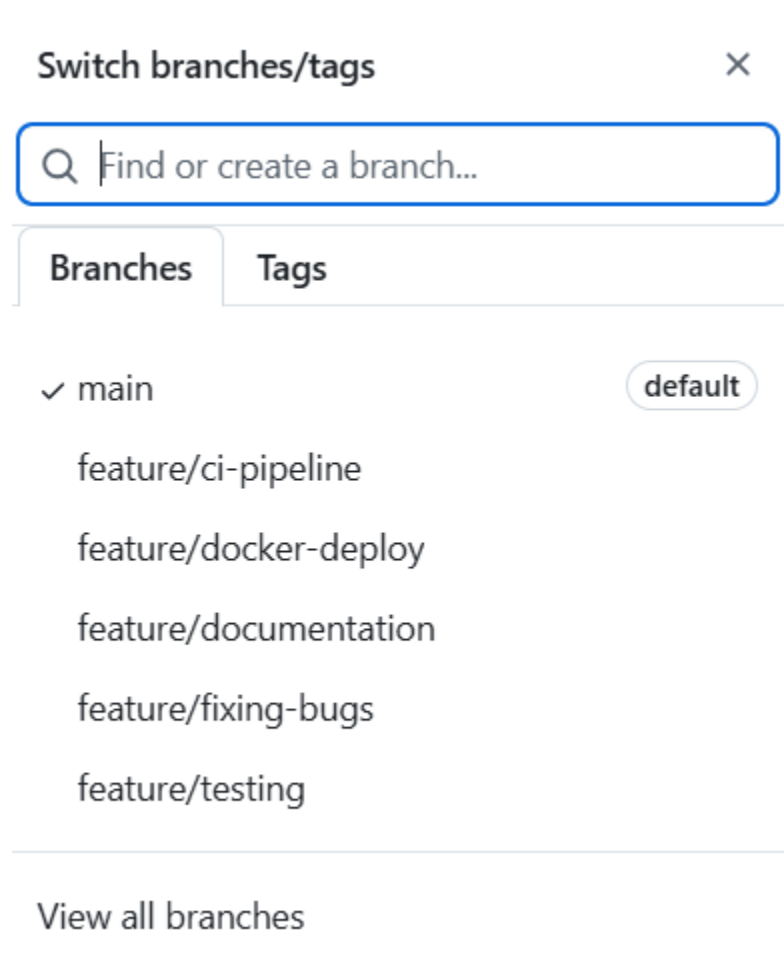
Team Role	Branch Name
Documentation	feature/documentation
QA/Testing	feature/testing
Bug Fixing	feature/bug-fixes
CI Pipeline	Feature/ci-pipeline
Docker and Deployment	feature/docker-deploy

Screenshots

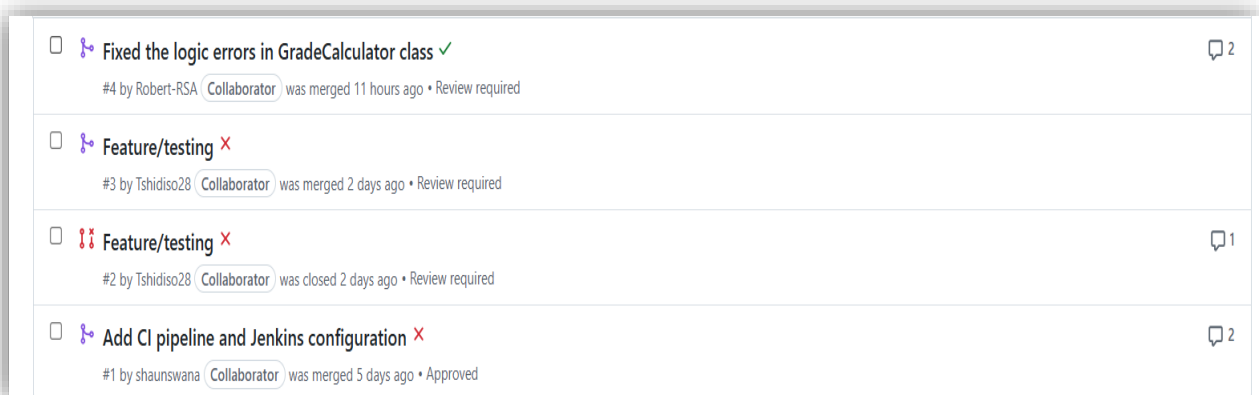
4.1 GitHub Repository



4.2 Feature Branches



4.3 Pull Requests



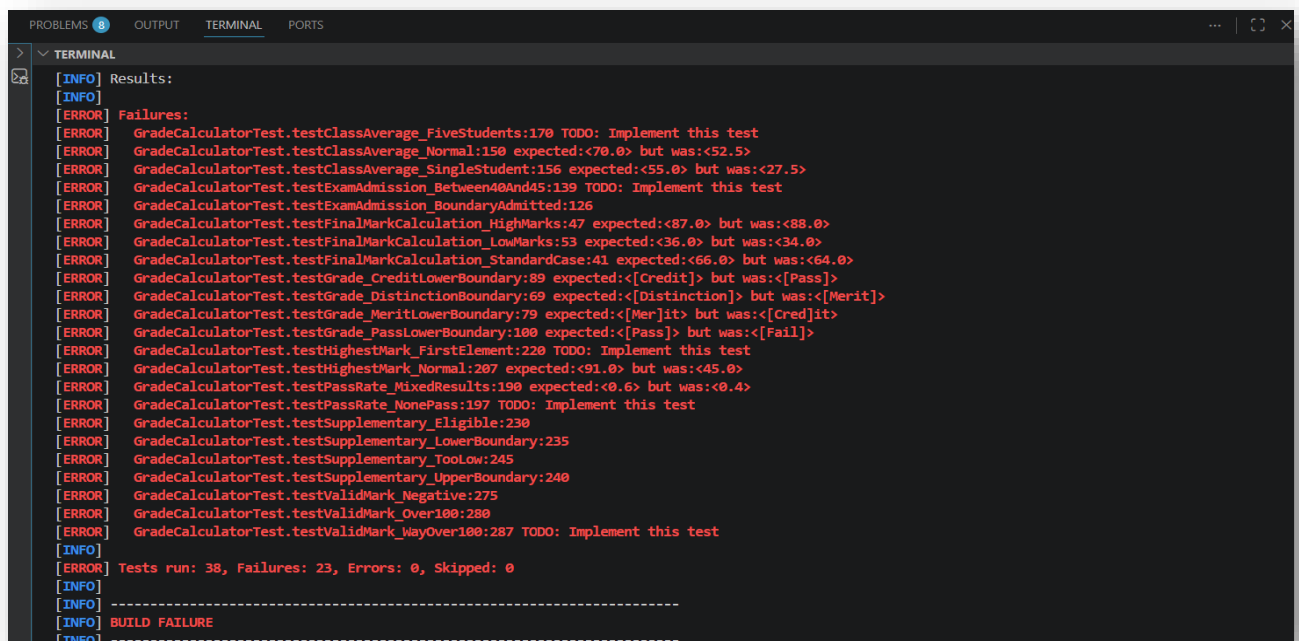
5. Build and Testing Process

Maven was used as the build tool for the project. We ran the commands **mvn clean package** and **mvn test** on the computer. The **mvn clean package** command removes old build files, compiles the project, runs the tests, and creates a WAR file. The **mvn test** command is used to run unit tests. When the tests were first executed, some of them failed, which showed that there were logic errors in the application.

These failed tests helped us identify problems and guided us in fixing the bugs in the system.

Screenshots

5.1 Maven Build Output



```
PROBLEMS 8 OUTPUT TERMINAL PORTS
> TERMINAL
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR] GradeCalculatorTest.testClassAverage_FiveStudents:170 TODO: Implement this test
[ERROR] GradeCalculatorTest.testClassAverage_Normal:150 expected:<70.0> but was:<52.5>
[ERROR] GradeCalculatorTest.testClassAverage_SingleStudent:156 expected:<55.0> but was:<27.5>
[ERROR] GradeCalculatorTest.testExamAdmission_Between40And45:139 TODO: Implement this test
[ERROR] GradeCalculatorTest.testExamAdmission_BoundaryAdmitted:126
[ERROR] GradeCalculatorTest.testFinalMarkCalculation_HighMarks:47 expected:<87.0> but was:<88.0>
[ERROR] GradeCalculatorTest.testFinalMarkCalculation_LowMarks:53 expected:<36.0> but was:<34.0>
[ERROR] GradeCalculatorTest.testFinalMarkCalculation_StandardCase:41 expected:<66.0> but was:<64.0>
[ERROR] GradeCalculatorTest.testGrade_CreditLowerBoundary:89 expected:<[Credit]> but was:<[Pass]>
[ERROR] GradeCalculatorTest.testGrade_DistinctionBoundary:69 expected:<[Distinction]> but was:<[Merit]>
[ERROR] GradeCalculatorTest.testGrade_MeritLowerBoundary:79 expected:<[Merit]> but was:<[Credit]>
[ERROR] GradeCalculatorTest.testGrade_PassLowerBoundary:100 expected:<[Pass]> but was:<[Fail]>
[ERROR] GradeCalculatorTest.testHighestMark_FirstElement:220 TODO: Implement this test
[ERROR] GradeCalculatorTest.testHighestMark_Normal:207 expected:<91.0> but was:<45.0>
[ERROR] GradeCalculatorTest.testPassRate_MixedResults:190 expected:<0.6> but was:<0.4>
[ERROR] GradeCalculatorTest.testPassRate_NonePass:197 TODO: Implement this test
[ERROR] GradeCalculatorTest.testSupplementary_Eligible:230
[ERROR] GradeCalculatorTest.testSupplementary_LowerBoundary:235
[ERROR] GradeCalculatorTest.testSupplementary_TooLow:245
[ERROR] GradeCalculatorTest.testSupplementary_UpperBoundary:240
[ERROR] GradeCalculatorTest.testValidMark_Negative:275
[ERROR] GradeCalculatorTest.testValidMark_Over100:280
[ERROR] GradeCalculatorTest.testValidMark_WayOver100:287 TODO: Implement this test
[INFO] Tests run: 38, Failures: 23, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
```

6. Bug Identification and Resolution

The application had several logic errors in the GradeCalculator. Java file. The QA and bug fixing team found these errors during unit testing and fixed them according to the required business rules.

6.1 Bug Report Table

Bug	Problem Identified	Expected Behaviour	Fix Applied
1	Final mark formula weights swapped	Semester 40% ,Exam 60%	Correct admission logic
2	Incorrect grade boundaries	Correct grade ranges applied	Fixed average calculation
3	Incorrect exam admission threshold	Admission at ≥ 40	Corrected admission logic
4	Class average off-by-one error	Divide by correct array length	Fixed average calculation
5	Incorrect pass rate calculation	Pass mark ≥ 50	Corrected pass rate logic
6	Highest mark method returned lowest value	Highest mark returned	Corrected comparison operator
7	Incorrect supplementary range	Supplementary between 45-49	Corrected supplementary logic
8	Invalid mark validation	Only 0-100 valid	Added proper validation
9	Student report admission logic inverted	Correct admission display	Fixed report condition
10	TODO test incomplete	Tests should execute correctly	Implemented missing unit tests

6.2 Before Bug Fixes

During initial testing, multiple unit test failed due to intentional logic errors in the application

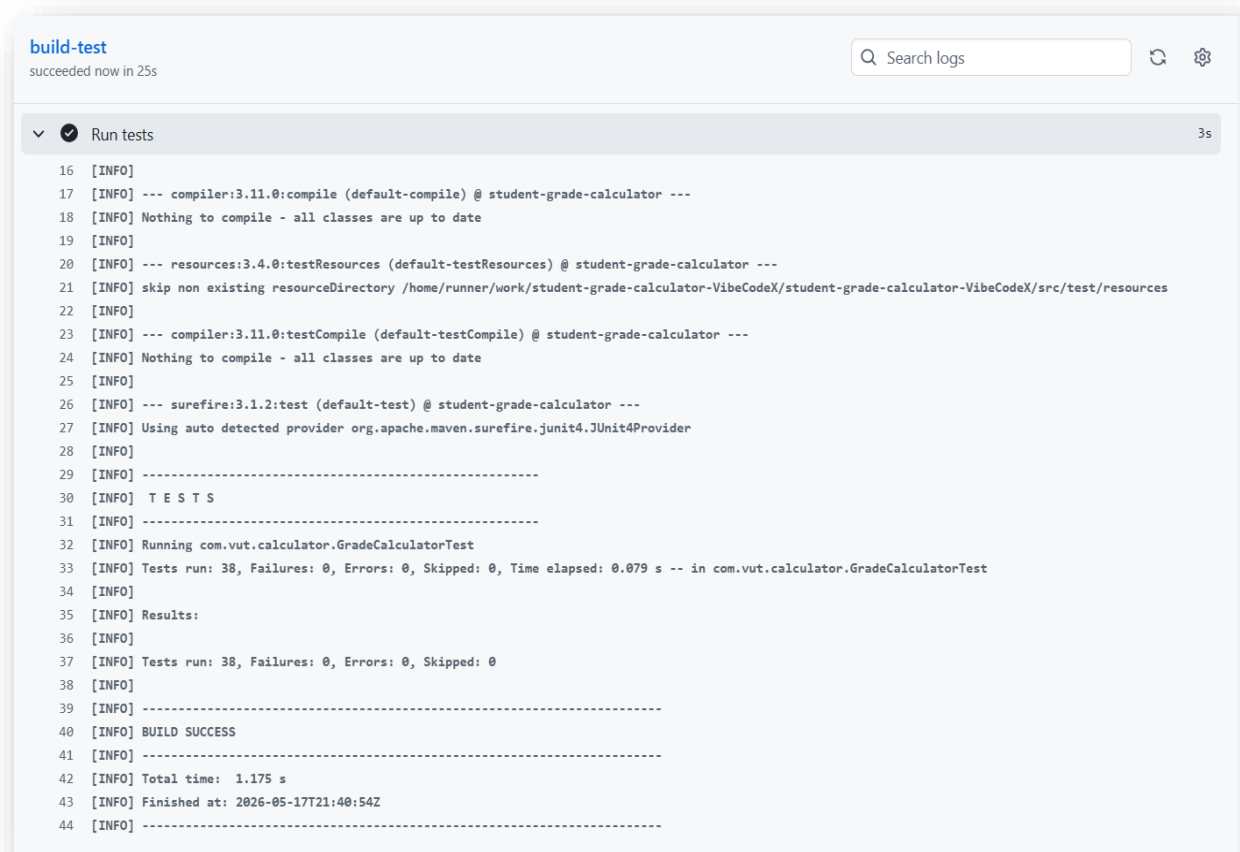
Screenshots

```
3080 Error: Failures:
3081 Error: GradeCalculatorTest.testClassAverage_FiveStudents:170 TODO: Implement this test
3082 Error: GradeCalculatorTest.testClassAverage_Normal:150 expected:<70.0> but was:<52.5>
3083 Error: GradeCalculatorTest.testClassAverage_SingleStudent:156 expected:<55.0> but was:<27.5>
3084 Error: GradeCalculatorTest.testExamAdmission_Between40And45:139 TODO: Implement this test
3085 Error: GradeCalculatorTest.testExamAdmission_BoundaryAdmitted:126
3086 Error: GradeCalculatorTest.testFinalMarkCalculation_HighMarks:47 expected:<87.0> but was:<88.0>
3087 Error: GradeCalculatorTest.testFinalMarkCalculation_LowMarks:53 expected:<36.0> but was:<34.0>
3088 Error: GradeCalculatorTest.testFinalMarkCalculation_StandardCase:41 expected:<66.0> but was:
    <64.0>
3089 Error: GradeCalculatorTest.testGrade_CreditLowerBoundary:89 expected:<[Credit]> but was:<[Pass]>
3090 Error: GradeCalculatorTest.testGrade_DistinctionBoundary:69 expected:<[Distinction]> but was:
    <[Merit]>
3091 Error: GradeCalculatorTest.testGrade_MeritLowerBoundary:79 expected:<[Mer]it> but was:<[Cred]it>
3092 Error: GradeCalculatorTest.testGrade_PassLowerBoundary:100 expected:<[Pass]> but was:<[Fail]>
3093 Error: GradeCalculatorTest.testHighestMark_FirstElement:220 TODO: Implement this test
3094 Error: GradeCalculatorTest.testHighestMark_Normal:207 expected:<91.0> but was:<45.0>
3095 Error: GradeCalculatorTest.testPassRate_MixedResults:190 expected:<0.6> but was:<0.4>
3096 Error: GradeCalculatorTest.testPassRate_NonePass:197 TODO: Implement this test
3097 Error: GradeCalculatorTest.testSupplementary_Eligible:230
3098 Error: GradeCalculatorTest.testSupplementary_LowerBoundary:235
3099 Error: GradeCalculatorTest.testSupplementary_TooLow:245
3100 Error: GradeCalculatorTest.testSupplementary_UpperBoundary:240
3101 Error: GradeCalculatorTest.testValidMark_Negative:275
3102 Error: GradeCalculatorTest.testValidMark_Over100:280
3103 Error: GradeCalculatorTest.testValidMark_WayOver100:287 TODO: Implement this test
3104 [INFO]
3105 Error: Tests run: 38, Failures: 23, Errors: 0, Skipped: 0
```

6.3 After Bug Fixes

After correcting the identified logic errors and implementing missing TODO tests, all unit tests passed successfully.

Screenshot



The screenshot shows a 'build-test' window with a search bar and a refresh icon. Below the header, a section titled 'Run tests' with a checkmark icon and '3s' indicates the test duration. The main area displays a log of build and test output. The log shows that the compiler and test compiler are up to date, and the test suite 'com.vut.calculator.GradeCalculatorTest' passed with 38 tests, 0 failures, 0 errors, and 0 skipped tests. The build is marked as 'BUILD SUCCESS' and the total time is 1.175 s.

```
16 [INFO]
17 [INFO] --- compiler:3.11.0:compile (default-compile) @ student-grade-calculator ---
18 [INFO] Nothing to compile - all classes are up to date
19 [INFO]
20 [INFO] --- resources:3.4.0:testResources (default-testResources) @ student-grade-calculator ---
21 [INFO] skip non existing resourceDirectory /home/runner/work/student-grade-calculator-VibeCodeX/student-grade-calculator-VibeCodeX/src/test/resources
22 [INFO]
23 [INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ student-grade-calculator ---
24 [INFO] Nothing to compile - all classes are up to date
25 [INFO]
26 [INFO] --- surefire:3.1.2:test (default-test) @ student-grade-calculator ---
27 [INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
28 [INFO]
29 [INFO] -----
30 [INFO] T E S T S
31 [INFO] -----
32 [INFO] Running com.vut.calculator.GradeCalculatorTest
33 [INFO] Tests run: 38, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.079 s -- in com.vut.calculator.GradeCalculatorTest
34 [INFO]
35 [INFO] Results:
36 [INFO]
37 [INFO] Tests run: 38, Failures: 0, Errors: 0, Skipped: 0
38 [INFO]
39 [INFO] -----
40 [INFO] BUILD SUCCESS
41 [INFO] -----
42 [INFO] Total time: 1.175 s
43 [INFO] Finished at: 2026-05-17T21:40:54Z
44 [INFO] -----
```

7. GitHub Actions Continuous Integration

GitHub Actions was used to automate the Continuous Integration (CI) process for the Student Grade Calculator application.

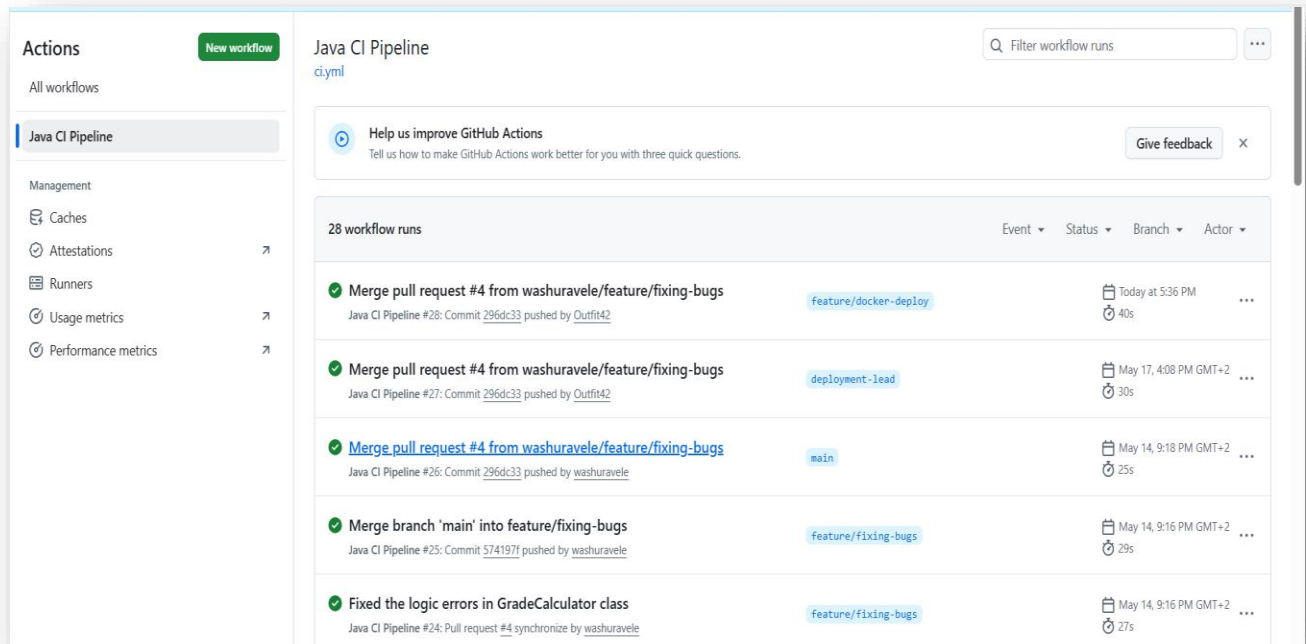
A workflow file called **ci.yml** was created inside the **.github/workflows** folders. The workflow automatically builds and test the application whenever new code is pushed to the repository or when a pull request is created. It checks the project source code, sets up JDK 11, builds the Maven project, and runs automated unit tests. Continuous Integration helped the group find errors early and made sure that new code changes did not break the application.

Screenshots

7.1 GitHub Actions Red Build

Status	Workflow Name	Commit	Author	Branch	Duration
Failed	Bug#9 for exam admission fixed	95ebc80	Robert-RSA	feature/fixing-bugs	23s
Failed	Bug#8 for mark validation fixed	80f235a	Robert-RSA	feature/fixing-bugs	25s
Failed	Bug #6 for highest mark fixed and Bug#7 for checking supplementar...	e842fc8	Robert-RSA	feature/fixing-bugs	22s
Failed	bug #4 for calculating pass rate fixwd	b4df442	Robert-RSA	feature/fixing-bugs	23s
Failed	Bud #4 for calculating average fixed	4f16617	Robert-RSA	feature/fixing-bugs	24s
Failed	Exam admission bug fixed	72704c3	Robert-RSA	feature/fixing-bugs	22s
Failed	Grade boundary logic fixed	249cca2	Robert-RSA	feature/fixing-bugs	28s
Failed	Merge pull request #3 from washuravele/feature/testing	f3d7075	washuravele	main	25s
Failed	Feature/testing		Tshidiso28	feature/testing	30s
Failed	Feature/testing			feature/testing	

7.2 GitHub Actions Green Build



The screenshot displays the GitHub Actions interface for a workflow named 'Java CI Pipeline'. The left sidebar shows the 'Actions' tab with a 'New workflow' button and a list of workflow categories: All workflows, Java CI Pipeline (selected), Management, Caches, Attestations, Runners, Usage metrics, and Performance metrics. The main area shows the 'Java CI Pipeline' workflow with a search bar and a 'Filter workflow runs' button. Below this is a 'Help us improve GitHub Actions' section. The '28 workflow runs' section lists the following runs:

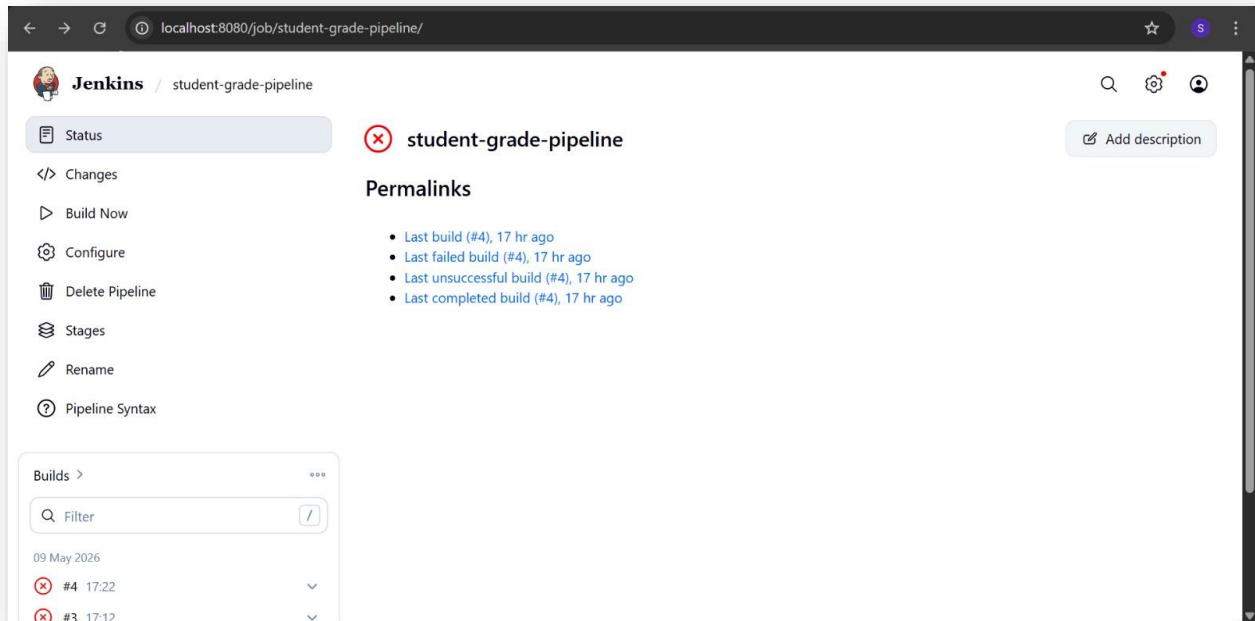
Event	Status	Branch	Actor
Merge pull request #4 from washuravele/feature/fixing-bugs	Success	feature/docker-deploy	Outfit42
Merge pull request #4 from washuravele/feature/fixing-bugs	Success	deployment-lead	Outfit42
Merge pull request #4 from washuravele/feature/fixing-bugs	Success	main	washuravele
Merge branch 'main' into feature/fixing-bugs	Success	feature/fixing-bugs	washuravele
Fixed the logic errors in GradeCalculator class	Success	feature/fixing-bugs	washuravele

8. Jenkins Pipeline Implementation

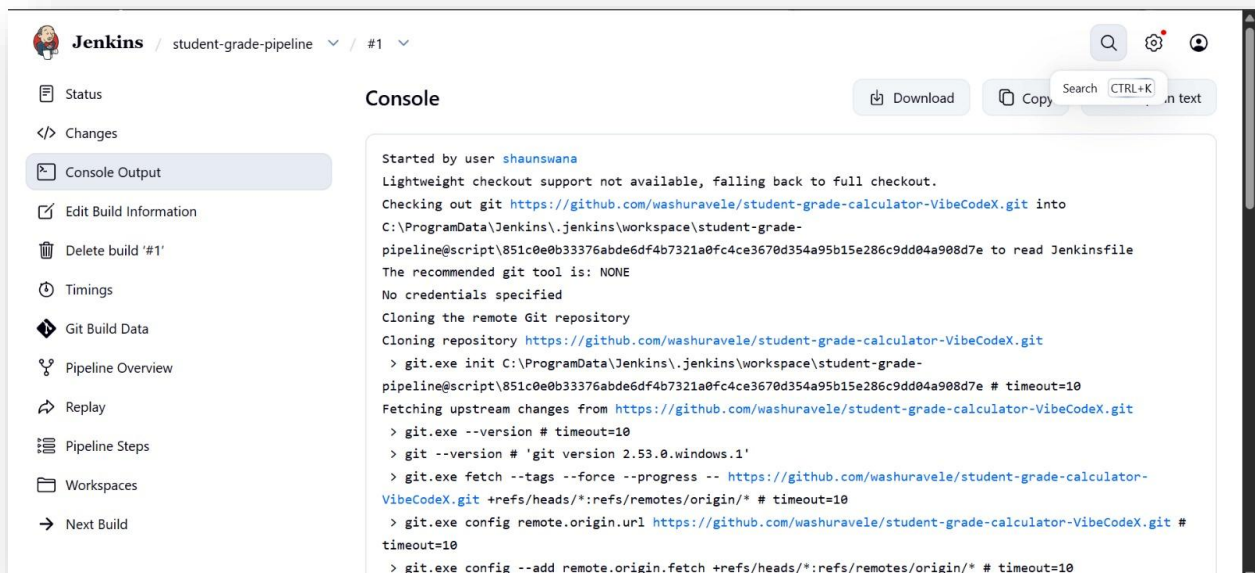
Jenkins was used as an additional Continuous Integration and Continuous Delivery (CI/CD) tool for the project. A Jenkins file was created using declarative pipeline syntax. The Jenkins pipeline included different stages such as the build stage, test stage, test report publishing, and WAR file archiving. These stages helped automate the process of building, testing, and storing the application files, making development faster and more reliable.

Screenshots

8.1 Jenkins Dashboard



8.2 Jenkins Console Output

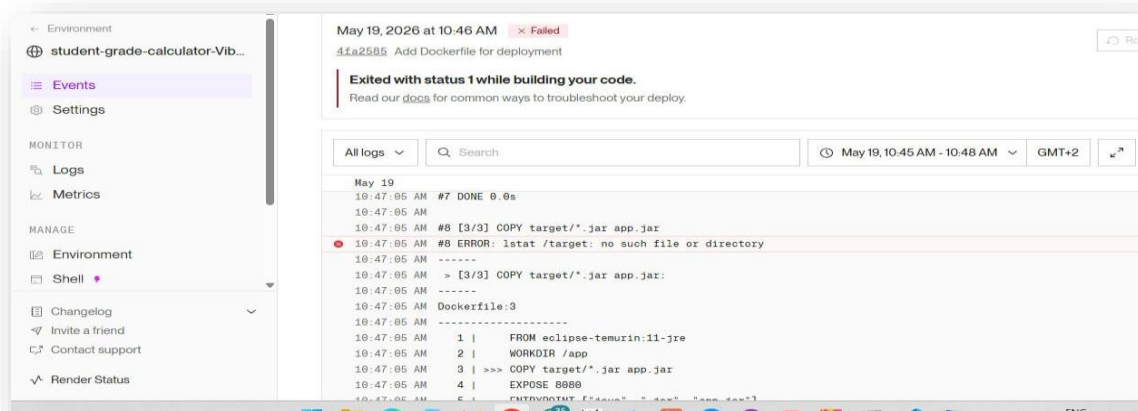


9. Docker Containerisation

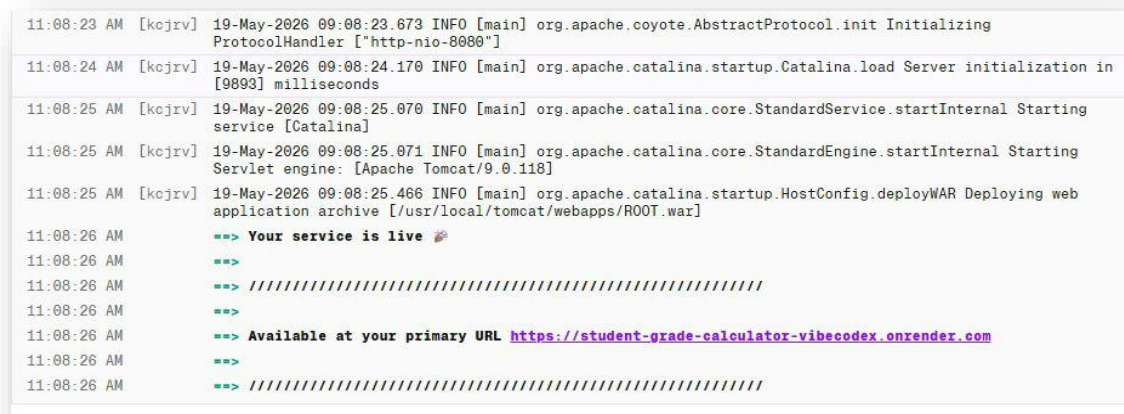
The application was containerised using Docker. A Dockerfile was created using Tomcat 9 with JDK 11 as the base image. The generated WAR file was added as the ROOT application in the Tomcat server. The Docker container used port 808 and included JVM memory limits that were suitable for the Render free tier environment. Using Docker made the application easier to deploy and run on different systems.

Screenshots

9.1 Docker Build



9.2 Running Docker Container

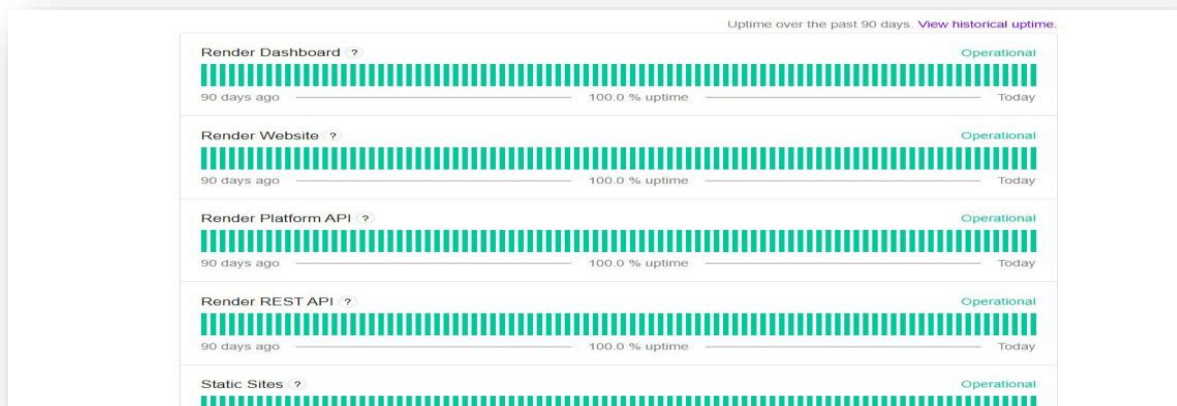
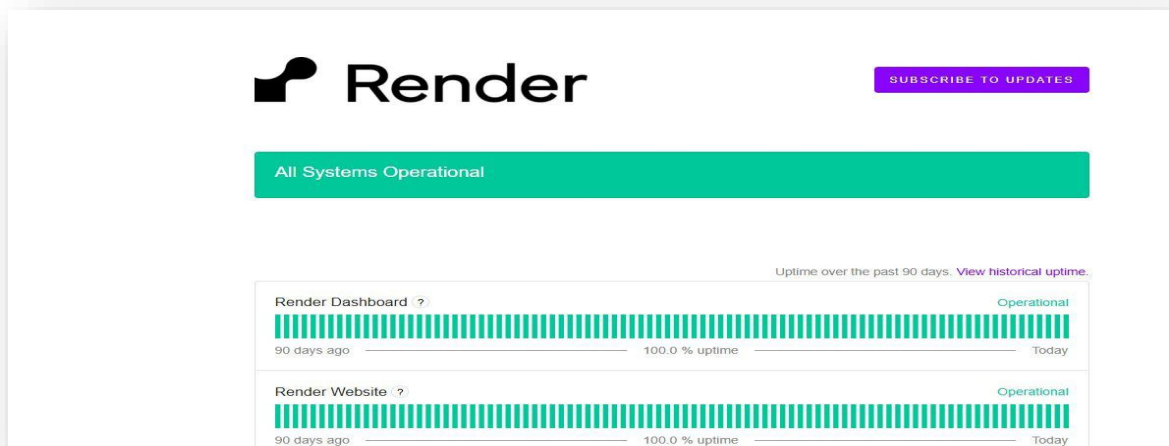


10. Deployment to Render.com

The application was deployed online using Render. A deployment workflow was created in GitHub Actions to automatically deploy the application using Render deploy hooks after successful testing. The workflow only runs when code is pushed to the main branch, uses GitHub secrets for secure deployment, and automatically start the deployment process on Render.

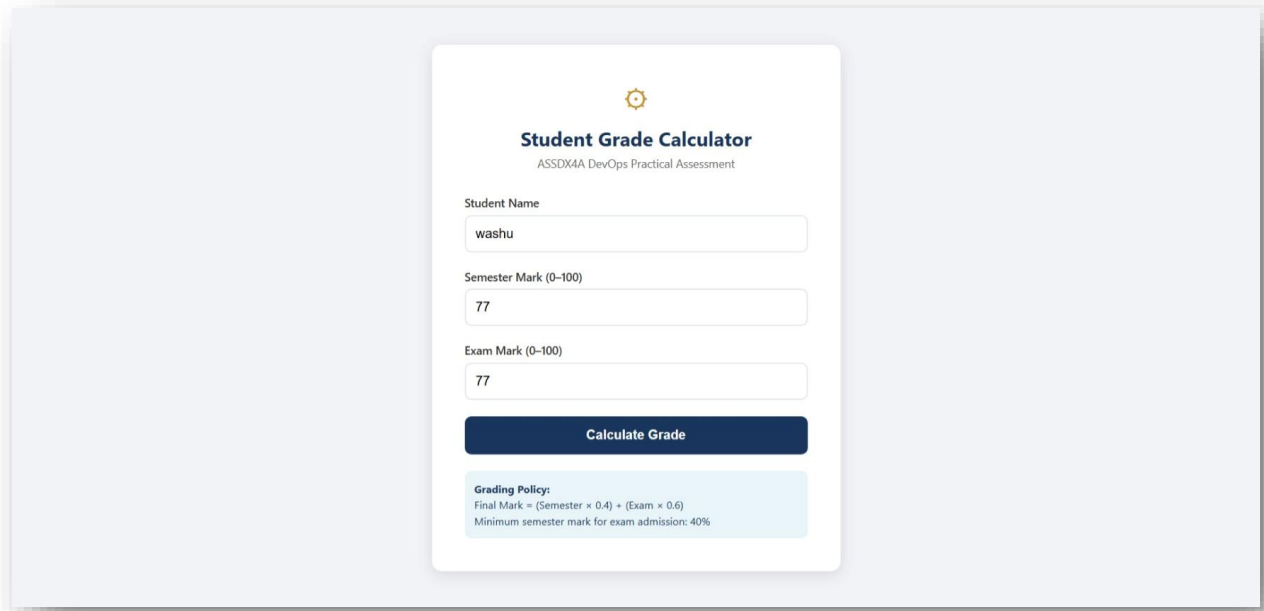
Screenshots

10.1 Render Dashboard



10.2 Live Running Application

Application Link: <https://student-grade-calculator-vibecodex.onrender.com>



The screenshot shows the 'Student Grade Calculator' application. It features a central white card on a light blue background. At the top of the card is a gear icon, followed by the title 'Student Grade Calculator' and the subtitle 'ASSDX4A DevOps Practical Assessment'. Below this are three input fields: 'Student Name' with the value 'washu', 'Semester Mark (0-100)' with the value '77', and 'Exam Mark (0-100)' with the value '77'. A dark blue button labeled 'Calculate Grade' is positioned below the input fields. At the bottom of the card, a light blue box contains the 'Grading Policy' section, which includes the formula 'Final Mark = (Semester × 0.4) + (Exam × 0.6)' and the text 'Minimum semester mark for exam admission: 40%'.

Student Grade Calculator
ASSDX4A DevOps Practical Assessment

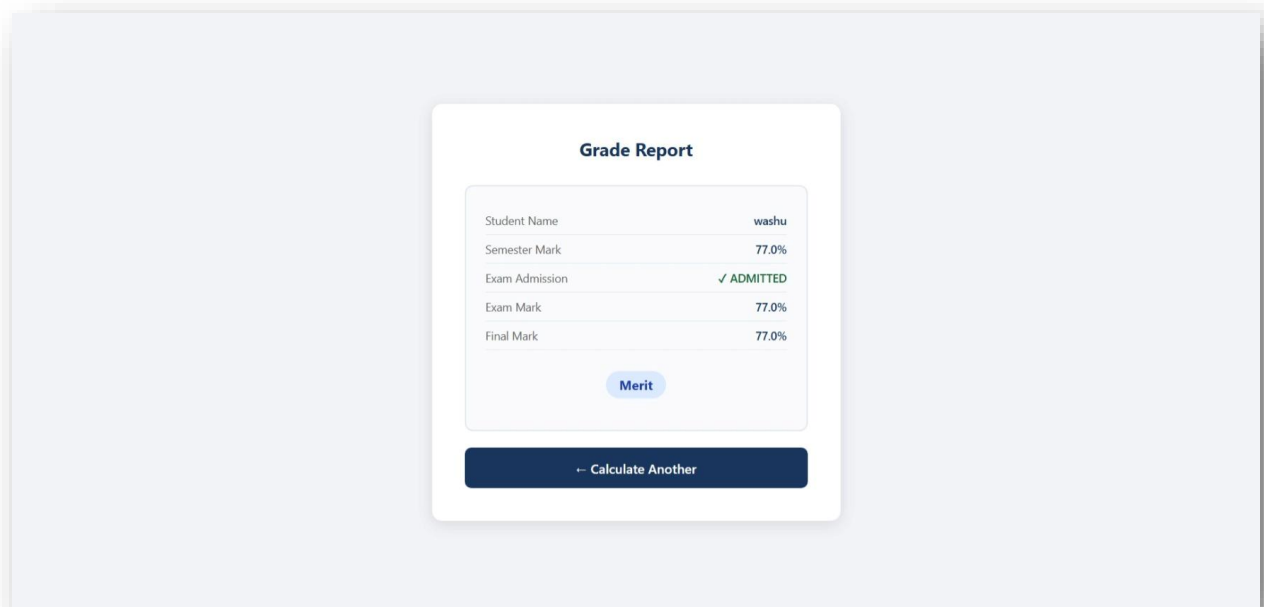
Student Name
washu

Semester Mark (0-100)
77

Exam Mark (0-100)
77

Calculate Grade

Grading Policy:
Final Mark = (Semester × 0.4) + (Exam × 0.6)
Minimum semester mark for exam admission: 40%



The screenshot shows the 'Grade Report' application. It features a central white card on a light blue background. The title 'Grade Report' is at the top. Below it is a table with student information. The table has two columns: the first column lists the fields (Student Name, Semester Mark, Exam Admission, Exam Mark, Final Mark) and the second column shows the values (washu, 77.0%, ✓ ADMITTED, 77.0%, 77.0%). Below the table is a blue button labeled 'Merit'. At the bottom of the card is a dark blue button labeled '← Calculate Another'.

Grade Report

Student Name	washu
Semester Mark	77.0%
Exam Admission	✓ ADMITTED
Exam Mark	77.0%
Final Mark	77.0%

Merit

← Calculate Another

11. Conclusion

This project gave practical experience in using modern DevOps tools and technologies. The assessment showed effective teamwork using GitHub, Continuous Integration using GitHub Actions and Jenkins, automated testing and bug fixing, Docker containerisation , and cloud deployment using Render. The successful use of the CI/CD pipeline improved software reliability, automation and deployment efficiency.